

Contents

□ Dynamic Programming — 2D / Grid — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	10
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	16
SECTION 20 — Senior Engineer Insights	16
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	16
SECTION 23 — Practice Roadmap	17
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	17

□ **Dynamic Programming — 2D / Grid — Complete Interview Handbook**

Pattern #23 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Dynamic Programming (2D/Grid) section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

2D/Grid Dynamic Programming extends 1D DP to problems whose state naturally requires **two indices** — most commonly literal grid coordinates (`row`, `col`), but also abstract pairs like “considering the first i elements of sequence A and the first j elements of sequence B.” $dp[i][j]$ represents the optimal value/count for the subproblem defined by both indices, built from a recurrence referencing smaller (i', j') pairs.

1.2 Why Was This Pattern Invented?

Some problems fundamentally have two independently-varying quantities that both affect the answer — a robot’s position on a grid (row AND column both matter), or comparing two separate sequences (position in sequence A AND position in sequence B both matter). A single index can’t capture this; 2D DP formalizes tracking both, with the recurrence typically referencing the cell above, to the left, or diagonally adjacent (or their sequence-pair analogs).

1.3 Real Intuition Behind The Pattern

Imagine a **robot navigating a city grid who can only move right or down** — the number of ways to reach any intersection (r, c) is exactly the sum of the ways to reach the intersection directly above it and the ways to reach the intersection directly to its left (since those are the only two places it could have come from). This is the canonical 2D DP recurrence.

1.4 Mental Model

“What are the (at most a few) cells/positions I could have come from to reach this one, and how do their answers combine to give mine?” Just like 1D DP, but now the “smaller subproblems” are indexed by two coordinates instead of one, and the fill order must respect dependencies in both dimensions.

1.5 Visual Explanation

Unique Paths (3x3 grid, robot moves only right/down, starting top-left):

```
dp[i][j] = dp[i-1][j] + dp[i][j-1]
dp[0][j] = 1 for all j (only one way: all rights)
dp[i][0] = 1 for all i (only one way: all downs)
```

	j=0	j=1	j=2
i=0:	1	1	1
i=1:	1	2	3
i=2:	1	3	6

```
dp[2][2] = dp[1][2] + dp[2][1] = 3 + 3 = 6 → 6 total paths
```

1.6 Simple Analogy

2D DP is like **filling in a spreadsheet where every cell’s value depends only on cells above and to the left (already computed)** — you fill row by row (or column by column), and by the time you reach any cell, everything it needs is already sitting there waiting.

1.7 When Should I Immediately Think About Using This Pattern?

- Literal grid problems: “unique paths,” “minimum path sum,” “maximal square.”

- Comparing **two sequences**: “longest common subsequence,” “edit distance” (though these are covered in depth in Pattern #25, DP-Strings, they’re fundamentally 2D DP).
- “Number of ways to...” over a 2D structure (grid or two-sequence comparison).
- Any brute-force recursion with **two changing parameters**, both needed to describe the subproblem.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“grid,” “matrix,” “m x n”	Literal 2D grid DP
“unique paths,” “minimum path sum”	Grid traversal DP
“two strings/arrays,” “common subsequence”	Sequence-pair 2D DP
“maximal square/rectangle”	Grid-based 2D DP with a specialized recurrence
“robot,” “obstacles”	Grid DP with additional constraint handling

2.2 Hidden Hints

A brute-force recursive solution with **two changing parameters** in its function signature (`solve(i, j)`) — not one — is the definitive signal that 2D DP is needed, rather than trying to force it into a 1D formulation.

2.3 Interview Clues

Interviewer explicitly gives a grid/matrix as input, or describes a problem comparing two separate sequences character-by-character or element-by-element.

2.4 Common Trick Words

“Obstacles” (requires explicit blocking logic in the recurrence), “at most k obstacles removed” (adds a third dimension — obstacles used — turning this into 3D DP), “minimum falling path” (grid DP with a specific “from which of 3 cells above” recurrence).

2.5 What Interviewers Expect

Correct recurrence direction (which cells feed into `dp[i][j]`), correct boundary/edge handling (first row/column often need special base-case treatment), and recognition of when space can be optimized from $O(m \times n)$ to $O(n)$ (or $O(\min(m, n))$) since most grid DP recurrences only reference the current and previous row.

2.6 When NOT To Use This Pattern

- The problem’s state only genuinely needs **one** index — forcing an unnecessary second dimension adds complexity without benefit; reconsider whether 1D DP (Pattern #22) suffices.
- The problem needs a **third** dimension (e.g., “items considered, capacity used, AND a third constraint”) — that likely needs 3D DP or a more specialized Knapsack variant (Pattern #24).
- Movement/comparison isn’t actually constrained to “from above/left” (or sequence-position analogs) — if arbitrary jumps are allowed, the DAG-like dependency structure may require a different traversal order (e.g., topological sort-guided DP) rather than simple row/column iteration.

SECTION 3 — Decision Framework

Does the brute-force recursion have TWO changing parameters (both needed to define the subproblem)?

Yes

Is this a LITERAL GRID (row, col) or a TWO-SEQUENCE COMPARISON (i in seq A, j in seq B)?

Grid Sequences

USE GRID DP USE SEQUENCE-PAIR DP

(dp[i][j] from above/left) (dp[i][j] from dp[i-1][j-1], dp[i-1][j], dp[i][j-1] - see Pattern #25 for string-specific depth)

Does the recurrence ONLY reference the immediately previous row/column?

Yes → SPACE-OPTIMIZE to $O(n)$ (1D rolling array) instead of full $O(m \times n)$

No

Does the problem need a THIRD dimension (e.g., capacity, obstacles-used)?

Yes → Consider 3D DP or a KNAPSACK-style formulation (Pattern #24) instead

Why: The defining characteristic of 2D DP is that exactly two independent quantities are needed to fully describe a subproblem's state — recognizing whether those two quantities are grid coordinates or sequence positions determines the specific recurrence shape, while the space-optimization opportunity (most 2D DP only needs the previous row) is a near-universal follow-up.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: Just as in 1D DP, correctness rests on **optimal substructure** (the optimal answer for (i, j) is composed from optimal answers to strictly “smaller” (i', j') pairs) and **overlapping subproblems** (many different paths through the grid/comparison reach the same (i, j) state, so caching avoids exponential re-computation). The two-dimensional nature just means the “smaller” relation is now a partial order over pairs (e.g., $i' < i$ and $j' < j$, with at least one strictly smaller), rather than a simple linear order.

Intuitive: For a grid where movement is only right/down, any cell (i, j) can only have been reached from $(i-1, j)$ (moved down) or $(i, j-1)$ (moved right) — exhaustively and without overlap counting the same path twice, which is exactly why $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ correctly counts all distinct paths.

Correctness Proof (by strong induction on $i+j$, the “distance” from the origin): *Base case:* $dp[0][0]$ (and the first row/column, reachable via only one direction) are directly known without recursion. *Inductive hypothesis:* assume $dp[i'][j']$ is correct for all $i'+j' < i+j$. *Inductive step:* $dp[i][j]$'s recurrence references only cells with strictly smaller $i'+j'$ (e.g., $(i-1, j)$ and $(i, j-1)$, both with sum $i+j-1$), which are correct by the inductive hypothesis; combining them per the problem's specific rule (sum for counting paths, min/max for path-cost problems) yields the correct $dp[i][j]$. *Termination:* by induction, $dp[m-1][n-1]$ (or wherever the “final answer” cell is) is correct. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. **Define the state:** precisely state what $dp[i][j]$ represents (e.g., “number of ways to reach cell (i,j) ” or “length of LCS using the first i chars of A and first j chars of B”).
2. **Define the recurrence:** express $dp[i][j]$ in terms of $dp[i-1][j]$, $dp[i][j-1]$, and/or $dp[i-1][j-1]$.
3. **Define base cases:** typically the first row and first column (grid) or $dp[0][*]/dp[*][0]$ (sequence comparison, representing empty-prefix cases).
4. **Choose fill order:** row by row, left to right (ensures dependencies are ready).
5. **Space-optimize:** if only the previous row is needed, use a 1D rolling array instead of the full 2D table.

5.2 General Template — Grid DP (Unique Paths style)

```
function uniquePaths(m, n):
    dp = 2D array of size m x n
    for i in range(0, m): dp[i][0] = 1
    for j in range(0, n): dp[0][j] = 1

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]
```

5.3 General Template — Sequence-Pair DP (LCS style, previewed here; full depth in Pattern #25)

```
function longestCommonSubsequence(A, B):
    m, n = length(A), length(B)
    dp = 2D array of size (m+1) x (n+1), all zeros

    for i in range(1, m+1):
        for j in range(1, n+1):
            if A[i-1] == B[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

5.4 Space-Optimized Template (Rolling Row)

```
function uniquePathsOptimized(m, n):
    prevRow = array of size n, all 1s (represents dp[0][*])
    for i in range(1, m):
        currentRow = array of size n
        currentRow[0] = 1 # dp[i][0] = 1
        for j in range(1, n):
            currentRow[j] = currentRow[j-1] + prevRow[j]
        prevRow = currentRow
    return prevRow[n-1]
```

5.5 Interview Thinking Process

1. "This has two independent varying quantities — I'll define $dp[i][j]$ precisely in terms of both."
 2. "I'll figure out which cells feed into $dp[i][j]$ — usually above, left, or diagonal, depending on whether this is a grid or sequence comparison."
 3. "I'll carefully handle the first row/column (or empty-prefix cases) as base cases, since they often have a different recurrence than the general case."
 4. "I'll fill the table row by row, left to right, ensuring dependencies are always ready."
 5. "I'll check if only the previous row is ever needed, and space-optimize to $O(n)$ if so."
-

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(m \times n)$	$O(m \times n)$, reducible to $O(\min(m,n))$ with rolling array	Every cell computed once, each in $O(1)$ (or $O(k)$ if the recurrence references more than a constant number of cells)
Average Case	$O(m \times n)$	$O(m \times n)$ or $O(n)$ optimized	Deterministic per-cell cost
Best Case	$O(m \times n)$ (must fill every reachable cell in general)	$O(n)$ optimized	Even simple grids require full table computation in most formulations
Amortized	$O(m \times n)$ total — each cell computed exactly once	$O(n)$ optimized	This is the entire benefit versus exponential brute-force path enumeration

SECTION 7 — Edge Cases

Edge Case	Example	Handling
1×1 grid	Single cell	$dp[0][0]$ is the trivial base case and also the final answer
Single row or single column	$1 \times n$ or $m \times 1$ grid	Only one direction of movement is possible — degenerates to a simple 1D case naturally
Obstacles blocking cells	"Unique Paths II"	Set $dp[i][j] = 0$ explicitly for obstacle cells, and ensure this propagates correctly (an obstacle cell contributes 0 to any cell depending on it)

Edge Case	Example	Handling
Empty sequences (LCS/Edit Distance)	A="" or B=""	dp[0][j] and dp[i][0] represent comparing against an empty prefix — must be initialized correctly, not left as arbitrary defaults
Negative numbers (Minimum Path Sum variants)	Grid contains negative values	The recurrence (min/max) still works correctly; no special-casing needed unless the problem has additional constraints (e.g., “path sum must stay non-negative throughout”)
Very large grids ($10^3 \times 10^3+$)	Memory constraints	Space-optimization to O(n) rolling array becomes essential, not just a nice-to-have
Starting/ending obstacle	Blocked start or end cell	If the start or end cell itself is blocked, the answer is immediately 0 — handle as an explicit special case before the main loop

Common mistakes: forgetting to correctly initialize the first row/column as base cases (assuming a default zero-initialization is always correct, which it isn’t for “number of ways” problems where the first row/column should typically be 1, not 0); incorrect obstacle propagation (forgetting that a blocked cell should have 0 ways, which then correctly propagates to cells depending on it without any special-case logic beyond the initial 0 assignment).

SECTION 8 — Pros & Cons

Advantages: Correctly and efficiently solves problems with two genuinely independent state dimensions; converts exponential path-enumeration into polynomial $O(m \times n)$ time; often space-optimizable to $O(\min(m, n))$. **Disadvantages:** $O(m \times n)$ can still be prohibitively large for very large grids/sequences (e.g., both dimensions in the millions); requires correctly identifying the recurrence direction and base cases, which is more involved than 1D DP. **Trade-offs:** Full 2D table (simpler to reason about, easier for path reconstruction) vs. space-optimized rolling array ($O(\min(m, n))$ space, but loses the ability to reconstruct the actual optimal path without additional bookkeeping). **Limitations:** Doesn’t scale to three or more independent state dimensions without further generalization (3D+ DP, which grows expensive quickly). **Inefficient when:** the state genuinely only needs one dimension (unnecessary 2D formulation adds complexity without benefit), or when the grid/sequence sizes are too large for even $O(m \times n)$ to be feasible within time limits.

SECTION 9 — Real World Applications

Domain	Application
Bioinformatics	DNA/protein sequence alignment (Needleman-Wunsch, Smith-Waterman algorithms) are directly 2D DP over two sequences
Version Control (Git/diff tools)	Computing minimal edit differences between file versions uses 2D DP (Edit Distance / LCS family)
Spell Checkers	Edit distance computation for “did you mean” suggestions
Robotics/Game Development	Grid-based pathfinding cost computation (minimum path sum, obstacle-avoiding path counting)
Image Processing	Maximal square/rectangle detection in binary images (e.g., finding the largest all-white region)
Natural Language Processing	Sequence alignment for machine translation evaluation metrics (e.g., BLEU score components)
Financial Systems	Two-dimensional resource allocation optimization (budget across time periods AND categories)
Logistics/Warehouse Systems	Grid-based optimal picking-path computation in warehouse layouts
Text Diffing Tools	Computing minimal insert/delete/substitute operations between text versions
Compression Algorithms	Certain dictionary-based compression schemes use longest-common-substring-style 2D DP internally

SECTION 10 — Interview Strategy

How seniors answer: They identify the two independent state dimensions explicitly, precisely define $dp[i][j]$ ’s meaning, correctly derive base cases for the first row/column (or empty-prefix cases), and proactively mention the space-optimization opportunity to $O(\min(m,n))$.

How juniors answer: They sometimes attempt to force a 2D problem into a 1D formulation (losing critical state information), or they get the base-case initialization wrong (e.g., initializing “number of ways” grids to 0 instead of 1 for the first row/column).

Typical follow-ups: “Can you optimize the space from $O(m \times n)$ to $O(n)$?” “What if there are obstacles?” “How would you reconstruct the actual optimal path, not just its value/count?” “How does this change if movement is allowed in more directions (diagonal, etc.)?”

Optimization questions: “Can you solve this with $O(\min(m,n))$ space instead of $O(\max(m,n))$?” (iterate over the smaller dimension as the “row” dimension in the rolling array).

SECTION 11 — Pattern Variations

Variation	Description	Example
Path Counting	$dp[i][j]$ = number of ways to reach (i,j)	Unique Paths, Unique Paths II (with obstacles)
Path Cost Optimization	$dp[i][j]$ = min/max cost path to (i,j)	Minimum Path Sum, Dungeon Game
Sequence Comparison	$dp[i][j]$ compares prefixes of two sequences	Longest Common Subsequence, Edit Distance (full depth in Pattern #25)
Maximal Region Detection	$dp[i][j]$ = size of the largest valid region ending at (i,j)	Maximal Square, Maximal Rectangle
Triangle/Irregular Grid DP	Non-rectangular grid traversal	Triangle (minimum path sum in a triangular grid)
Interval DP (2D over start/end of a range)	$dp[i][j]$ = best value for the subrange [i,j]	Matrix Chain Multiplication, Burst Balloons (technically interval DP, related family)
2D DP with Additional State (3D in practice)	Obstacles removed, k moves remaining, etc.	Cherry Pickup, Out of Boundary Paths

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
1D DP	Single-index state	State genuinely needs only one dimension
Knapsack DP	Also 2D (item index + capacity), but with a specific include/exclude choice structure per item	Problem is about selecting items subject to a capacity constraint (see Pattern #24 for full depth)
Interval DP	State is $dp[i][j]$ representing a subrange [i, j], filled by increasing range length, not row-by-row	Problem is naturally about optimal ways to combine/split a contiguous range
Graph Shortest Path	Grid can be modeled as a graph and solved via BFS/Dijkstra's if costs are more complex than simple right/down movement	Movement isn't restricted to a simple DP-friendly direction (e.g., arbitrary weighted edges)

Comparison Table

Aspect	2D Grid DP	1D DP	Knapsack DP
State dimensions	2 (row, col or two sequence positions)	1	2 (item index, capacity)
Typical complexity	$O(m \times n)$	$O(n)$	$O(n \times \text{capacity})$
Space optimization	Often to $O(\min(m,n))$	Often to $O(1)$	Often to $O(\text{capacity})$

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Simple grid path counting	Unique Paths, Minimum Path Sum
Medium	Obstacles, sequence comparison basics	Unique Paths II, Longest Common Subsequence, Triangle
Hard	Maximal region detection, complex recurrences	Maximal Square, Edit Distance, Dungeon Game
Very Hard	Additional state dimensions, advanced combinatorial DP	Cherry Pickup, Out of Boundary Paths, Maximal Rectangle

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Unique Paths	Medium	Amazon, Meta, Google, Microsoft	Direct grid path-counting DP	Foundational 2D recurrence
2	Unique Paths II	Medium	Amazon, Meta	Grid DP with obstacle handling	Obstacle propagation logic
3	Minimum Path Sum	Medium	Amazon, Meta, Google, Microsoft	Grid DP minimizing path cost	Cost-minimization recurrence
4	Triangle	Medium	Amazon, Google	Irregular-grid 2D DP	Non-rectangular grid handling
5	Longest Common Subsequence	Medium	Amazon, Meta, Google, Microsoft	Sequence-pair 2D DP	Foundational sequence-comparison DP
6	Edit Distance	Hard	Amazon, Meta, Google, Microsoft	Sequence-pair 2D DP with 3 operations	Advanced sequence-comparison DP
7	Maximal Square	Medium	Amazon, Meta, Google	Grid DP with specialized min-of-three recurrence	Advanced grid recurrence design
8	Maximal Rectangle	Hard	Amazon, Meta, Google	Grid DP combined with histogram/stack technique	Cross-pattern (2D DP + Monotonic Stack)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
9	Dungeon Game	Hard	Amazon, Google	Reverse-direction grid DP (bottom-up from destination)	Reverse-fill-order DP
10	Cherry Pickup	Hard	Google, Amazon	2D DP with dual-path simultaneous tracking (effectively higher-dimensional)	Advanced multi-path DP
11	Out of Boundary Paths	Medium	Amazon, Google	Grid DP with move-count as an added dimension	Cross-pattern (2D DP + counting dimension)
12	Distinct Subsequences	Hard	Amazon, Google, Microsoft	Sequence-pair 2D DP (counting variant)	Counting-based sequence DP
13	Interleaving String	Medium	Amazon, Google	Sequence-pair 2D DP with three-way comparison	Advanced three-sequence DP
14	Regular Expression Matching	Hard	Amazon, Meta, Google, Microsoft	Sequence-pair 2D DP with pattern-matching recurrence	Advanced pattern-matching DP
15	Wildcard Matching	Hard	Amazon, Google, Microsoft	Sequence-pair 2D DP with wildcard handling	Advanced pattern-matching DP variant
16	Minimum ASCII Delete Sum for Two Strings	Medium	Amazon, Google	Sequence-pair 2D DP (cost-based variant of LCS)	Cost-based sequence DP
17	Delete Operation for Two Strings	Medium	Amazon, Google	Sequence-pair 2D DP (reduces to LCS)	Problem reduction to LCS
18	Shortest Common Supersequence	Hard	Google, Amazon	Sequence-pair 2D DP + reconstruction	DP + path reconstruction
19	Longest Palindromic Subsequence	Medium	Amazon, Google	Sequence-vs-itself 2D DP (interval-style)	Self-comparison DP variant

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
20	Palindromic Substrings (contrast)	Medium	Amazon, Meta	Contrast: interval DP or expand-around-center, related family	Cross-pattern awareness
21	Count Square Submatrices with All Ones	Medium	Amazon, Google	Grid DP (same recurrence family as Maximal Square)	Recurrence reuse recognition
22	Minimum Falling Path Sum	Medium	Amazon, Google	Grid DP with three-way “from above” recurrence	Multi-source recurrence design
23	Cherry Pickup II	Hard	Google	Advanced multi-robot grid DP (3D in practice)	Advanced multi-agent DP
24	Number of Paths with Max Score	Hard	Google, Amazon	Grid DP with dual-value tracking (count + max score)	Dual-value DP tracking
25	K Inverse Pairs Array (contrast, combinatorial DP)	Hard	Google	Contrast: combinatorial 2D DP, not grid-based	Pattern-boundary awareness
26	Uncrossed Lines (contrast, equivalent to LCS)	Medium	Amazon, Google	Direct LCS reduction	Problem-recognition mastery
27	Path With Maximum Gold	Medium	Google, Amazon	Contrast: Backtracking on a grid, not pure DP (revisit allowed complicates DP)	Pattern-boundary awareness
28	Minimum Cost to Merge Stones (contrast, interval DP)	Hard	Google	Contrast: interval DP, not simple grid DP	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
29	Stone Game II/III (contrast, game-theoretic DP)	Medium/Hard	Google	Contrast: game-theoretic DP, related but distinct recurrence style	Pattern-boundary awareness
30	Burst Balloons (contrast, interval DP)	Hard	Google, Amazon	Contrast: classic interval DP, not row/column grid DP	Pattern-boundary awareness

SECTION 15 — Common Mistakes

1. Incorrect base-case initialization for the first row/column (e.g., leaving them at 0 when they should be 1 for “number of ways” problems). *Fix:* explicitly reason through what the first row/column represents before coding.
2. Confusing the recurrence direction — mixing up which cells feed into $dp[i][j]$ (above/left for grids, diagonal for sequence matches). *Fix:* draw a small example grid/table and trace the dependency direction explicitly.
3. Forgetting obstacle/blocked-cell propagation — not setting $dp[i][j] = 0$ for a blocked cell, causing it to incorrectly contribute to downstream cells. *Fix:* always handle blocked cells as an explicit early-continue/zero-assignment in the loop.
4. Attempting a full $O(m \times n)$ table when the problem has very large dimensions, missing the $O(\min(m, n))$ space-optimization opportunity. *Fix:* always check whether the recurrence only needs the previous row.
5. Forcing a problem that genuinely needs a third dimension (e.g., “moves remaining”) into a plain 2D formulation, losing critical state. *Fix:* carefully verify exactly how many independent quantities the state actually needs.

Why people fail: 2D DP requires correctly managing two simultaneous indices and their interacting base cases, which is meaningfully harder to get right under time pressure than 1D DP — candidates often get the *shape* of the recurrence right but fumble the boundary conditions (first row/column, obstacle handling), producing code that works on the happy path but fails on boundary-heavy test cases.

SECTION 16 — Optimization Techniques

- **Time:** Already typically optimal at $O(m \times n)$ for genuinely 2D-stateful problems — no further asymptotic improvement usually possible without additional problem-specific structure.
- **Space:** Reduce from $O(m \times n)$ to $O(\min(m, n))$ by iterating over the larger dimension as the outer loop and maintaining only a rolling 1D array for the smaller dimension.
- **Readability:** Clearly comment which direction (above, left, diagonal) each term in the recurrence represents; separate base-case initialization from the main double-loop visually.
- **Interview performance:** Explicitly state the two independent state dimensions and the base-case reasoning before coding — this demonstrates the same rigor as 1D DP but scaled correctly to the higher-dimensional case.

SECTION 17 — Multiple Language Templates

Java

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j-1];
        }
    }
    return dp[n-1];
}
```

JavaScript

```
function uniquePaths(m, n) {
    const dp = new Array(n).fill(1);
    for (let i = 1; i < m; i++) {
        for (let j = 1; j < n; j++) {
            dp[j] += dp[j-1];
        }
    }
    return dp[n-1];
}
```

PHP

```
function uniquePaths(int $m, int $n): int {
    $dp = array_fill(0, $n, 1);
    for ($i = 1; $i < $m; $i++) {
        for ($j = 1; $j < $n; $j++) {
            $dp[$j] += $dp[$j - 1];
        }
    }
    return $dp[$n - 1];
}
```

Python

```
def unique_paths(m, n):
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[-1]
```

Go

```
func uniquePaths(m int, n int) int {
    dp := make([]int, n)
    for i := range dp {
        dp[i] = 1
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            dp[j] += dp[j-1]
        }
    }
    return dp[n-1]
}
```

C++

```
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j-1];
        }
    }
    return dp[n-1];
}
```

SECTION 18 — Dry Runs

Small Input

m=3, n=3 (matches §1.5, space-optimized version)

dp = [1,1,1] (represents row 0)
i=1: j=1: dp[1] += dp[0] → dp=[1,2,1]
 j=2: dp[2] += dp[1] → dp=[1,2,3] (now represents row 1: [1,2,3])
i=2: j=1: dp[1] += dp[0] → dp=[1,3,3]
 j=2: dp[2] += dp[1] → dp=[1,3,6] (now represents row 2: [1,3,6])
Result: dp[2] = 6 (matches the full-table dry run in §1.5)

Large Input (Conceptual)

For a 1000×1000 grid, the space-optimized version uses $O(1000)$ space and $O(10^6)$ time — versus a naive recursive enumeration of all paths, which would be combinatorially infeasible ($C(1998, 999)$, an astronomically large number).

Corner Case

m=1, n=1: dp=[1] initially, loop doesn't execute (m=1 means the outer loop range(1,1) is empty) → result = dp[0] = 1, correctly representing the single trivial path (staying at the start, which is also the end).

SECTION 19 — Advanced Concepts

- **Reverse-direction DP (Dungeon Game):** some grid problems are more naturally solved by filling the table **backward** (from the destination toward the source) when the “cost so far” framing doesn’t naturally compose forward (e.g., needing enough health to survive the *remaining* path, not the path so far) — recognizing when to reverse the fill direction is a key advanced skill.
 - **Sequence DP with 3 dimensions collapsing to 2 (Interleaving String):** comparing three strings’ interleaving can be reduced to 2D DP because the third index is always determined by the other two ($k = i + j$), a common trick for “seemingly 3D but actually 2D” problems.
 - **Diagonal/anti-diagonal fill order (Interval DP, related family):** some 2D DP problems (like Matrix Chain Multiplication or Burst Balloons) must be filled by increasing **range length** (a diagonal fill pattern) rather than row-by-row, because $dp[i][j]$ depends on $dp[i][k]$ and $dp[k][j]$ for various k between i and j , not on the immediately adjacent row/column.
 - **Path reconstruction:** to recover the actual optimal path/alignment (not just its value), maintain a parallel “choice” table recording which recurrence branch was taken at each cell, then backtrack from the final cell to the origin using these recorded choices.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize 2D DP as the natural extension of the 1D DP principle to problems with two genuinely independent state dimensions, and they’re fluent in recognizing when a seemingly complex multi-parameter problem actually collapses to fewer effective dimensions (like Interleaving String’s $k = i+j$ trick). They also know to proactively suggest the $O(\min(m,n))$ space optimization and to recognize when a problem needs a non-standard fill order (reverse, diagonal) rather than the default row-by-row approach. Interviewers evaluate whether a candidate can correctly manage the added complexity of two-dimensional base cases and dependencies without introducing boundary bugs — a meaningfully harder bar than 1D DP, and a strong differentiator at the Senior/Staff level.

SECTION 21 — Cheat Sheet

PATTERN: 2D / Grid Dynamic Programming

RECOGNIZE: grid/matrix input, two-sequence comparison, brute-force recursion with TWO changing parameters

TEMPLATE (grid path counting):

```
dp[i][0] = 1 for all i; dp[0][j] = 1 for all j
for i in range(1,m): for j in range(1,n): dp[i][j] = dp[i-1][j] + dp[i][j-1]
return dp[m-1][n-1]
```

TEMPLATE (sequence comparison, LCS-style):

```
dp[i][0] = dp[0][j] = 0
for i,j: if A[i-1]==B[j-1]: dp[i][j]=dp[i-1][j-1]+1
        else: dp[i][j]=max(dp[i-1][j], dp[i][j-1])
```

COMPLEXITY: $O(m \times n)$ time; $O(m \times n)$ space, reducible to $O(\min(m,n))$ with a rolling row

KEY PROOF: optimal substructure + overlapping subproblems, now indexed by a 2D partial order ($i+j$ strictly)

WATCH FOR: correct base-case initialization (first row/col), obstacle propagation, recurrence direction, s

DOESN'T APPLY WHEN: state needs only 1 dimension (use 1D DP), state needs 3+ dimensions (extend further o

SECTION 22 — Revision Notes (5-Minute Review)

- $dp[i][j]$ needs two independent indices — grid coordinates or two sequence positions.

- Grid DP: usually $dp[i][j]$ from above ($dp[i-1][j]$) and left ($dp[i][j-1]$).
- Sequence-pair DP: usually $dp[i][j]$ from diagonal ($dp[i-1][j-1]$, match case) plus above/left (mismatch case).
- Base cases: first row/column (grid) or empty-prefix rows/columns (sequences) — get these right explicitly.
- Space-optimize to $O(\min(m,n))$ with a rolling row when only the previous row is needed.
- Some problems need reverse or diagonal fill order instead of simple row-by-row (Dungeon Game, Interval DP).
- Path reconstruction requires a parallel “choice” table, not just the value table.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic grid path counting/cost	Unique Paths (62), Minimum Path Sum (64)
Intermediate	Obstacles, basic sequence comparison	Unique Paths II (63), Longest Common Subsequence (1143), Triangle (120)
Advanced	Complex recurrences, maximal regions	Edit Distance (72), Maximal Square (221), Distinct Subsequences (115)
Expert	Multi-dimensional, advanced pattern matching	Regular Expression Matching (10), Cherry Pickup (741), Shortest Common Supersequence (1092)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Above, Left, Diagonal” (ALD) — the three directions most 2D DP recurrences pull from.
 - **Visualization:** A **spreadsheet filled row by row**, where every cell only needs cells above and to the left, already computed.
 - **Recognition shortcut:** Grid input, or two sequences being compared, with a brute-force recursion needing two parameters → 2D DP.
-

SECTION 25 — Final Summary

2D/Grid Dynamic Programming extends the 1D DP principle to problems whose state genuinely requires two independent indices — grid coordinates or two sequence positions — building each $dp[i][j]$ from a small set of already-solved smaller subproblems (above, left, or diagonal), converting exponential path/alignment enumeration into $O(m \times n)$ polynomial time. The single most important thing to remember forever: **correctly initialize the first row and column (or empty-prefix cases) as base cases — this is where the vast majority of 2D DP bugs occur — and always check whether the recurrence only ever needs the immediately previous row, which lets you space-optimize from $O(m \times n)$ down to $O(\min(m,n))$ with a rolling array.**